

PROYECTO INTEGRADO FP

DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Alumno: Andrés Jesús Jurado Suárez
2024

Sumario

Introducción.....	2
Viabilidad.....	3
Requisitos de información.....	.4
Requisitos funcionales.....	.5
Otros requisitos.....	.6
Análisis del sistema de información.....	7
Gestión de Tareas.....	.9
Notificaciones.....	.9
Seguimiento Temporal de Tareas.....	.10
Definición de Interfaces de Usuario y guía de uso.....	11
Pantalla de nueva tarea.....	.13
Selección de categoría.....	.14
Selección de color.....	.15
Selección de fecha.....	.16
Tipos de tarea.....	.18
Interacciones con las tareas.....	.19
Ventana de calendario.....	.20
Manual de instalación.....	22
Conclusiones.....	23
Bibliografía.....	24

Introducción

HabbitReminder

Este proyecto se enfoca en una app para dispositivos móviles encargada de llevar la gestión de tareas a lo largo del tiempo. La aplicación te permitirá crear tareas para una fecha determinada y te avisará en el momento oportuno, de la misma.

La aplicación sirve como recordatorio de rutinas y hábitos del día a día, y llegar un seguimiento de estos a lo largo del tiempo. Al ser una app para dispositivos móviles siempre vamos a poder estar al tanto de nuestras rutinas, como por ejemplo pasear a tu mascota o ir a la compra.

También, la aplicación no requiere de software adicional, ya que todos sus datos se guardan en el mismo dispositivo.

Viabilidad

Descripción del sistema

Actualmente, los usuarios gestionan sus tareas mediante métodos manuales o aplicaciones genéricas básicas que no están optimizadas para sus necesidades específicas. Estos métodos pueden ser ineficientes.

El nuevo sistema proporcionará una solución digital específica para la gestión de tareas, con una interfaz intuitiva y funcionalidades como notificaciones de vencimiento, categorización de tareas y calendario de seguimiento de estas

El lenguaje que utilizaremos será Kotlin, y usaremos el Framework de Jetpack Compose, que cambia totalmente la metodología de trabajo. Es totalmente declarativo, lo que significa que describes tu IU si llamas a una serie de funciones que transforman datos en una jerarquía de IU, y si, en la IU hay cambios, toda la pantalla es recompuesta.

Para la gestión de los datos usaremos Room, que nos permitirá manejar los datos de forma local a través de SQLite.

Todo esto lo manejaremos desde el IDE de Android Studio.

Identificación de requisitos del sistema

Requisitos de información

Trabajaremos con una base de datos relacional en la que guardaremos todos los datos de las tareas, y los manejaremos, como hemos dicho anteriormente, mediante queries SQLite y usando Room como gestor de base de datos.

Para las tablas(Entidades en Room), constaremos de 3:

Task, que contiene estos campos:

1. ID → Entero y auto incremental
2. Nombre → Cadena, no nulo
3. Color → Cadena, nutable
4. Descripción → Cadena, nutable
5. Fecha → Long, no nulo
6. Margen → Long, no nulo
7. Próxima fecha → Long, no nulo
8. Cumplida →Entero, no nulo
9. Notificada →Entero, no nulo
10. Categoría →Cadena, nutable

Category, que contiene estos campos:

1. **Id → Entero, auto incremental**
2. **Emoji → Cadena, no nutable**

Task_Category, que contiene estos campos:

1. **IDTask → Entero, no nutable**
2. **IDCategory → Entero, no nutable**

Estas serían las entidades que utilizaremos para construir nuestra base de datos

El volumen de datos usado para nuestra aplicación no será muy elevado debido a la reducida cantidad de información que manejaremos en cada rutina/tarea, al ser una aplicación de uso individual.

Requisitos funcionales

- La aplicación debe tener activada las notificaciones, para poder recibir estas una vez llega la fecha de una tarea determinada.
- La aplicación debe permitir crear tareas
- La aplicación debe permitir a los usuarios editar tareas existentes.
- La aplicación debe permitir a los usuarios eliminar tareas.
- La aplicación debe permitir a los usuarios marcar tareas como completadas.
- La aplicación debe permitir a los usuarios establecer una fecha y hora límite para cada tarea.
- La aplicación debe permitir a los usuarios agregar descripciones detalladas a las tareas.
- La aplicación debe permitir a los usuarios etiquetar tareas con palabras clave

- La aplicación debe permitir a los usuarios elegir entre diferentes temas de interfaz (modo claro y oscuro).
- La aplicación debe incluir una sección de ayuda con preguntas frecuentes y tutoriales.

Otros requisitos

La versión mínima del SDK de Android es 24, eso significa que esta aplicación solo funcionará en dispositivos que ejecuten Android 7.0 Nougat (API level 24) o versiones posteriores.

La aplicación debe de tener los permisos de notificaciones activados.

Descripción de la solución

Para poder llevar a cabo las distintas funciones de la aplicación, se ha tenido que implementar una serie de librerías para utilizar Room, otras para la inyección de dependencias, para la navegabilidad entre pantallas, para la acción de Swipe y distintas librerías intrínsecas de kotlin, tales como el uso del calendario y reloj.

Además necesitaremos la librería de WorkManager, para la gestión de las notificaciones en segundo plano

Equipo de trabajo

Para poder desarrollar la aplicación se ha usado un solo equipo de sobremesa cuyas características son 16 GB RAM, 1 TB HDD, 256 GB SSD M.2, y procesador intel core i5 13400 con un SO Windows 10.

También se han usado maquinas virtuales de smartphone en el IDE Android Studio,

Estudio del coste del proyecto

El coste será calculado en base a las horas dedicadas al proyecto y la investigación necesaria para poder desarrollar todas las funcionalidades.

Suponiendo que se ha trabajado semanalmente una media de 26 horas y cada hora sería cobrada a 10€, si el proyecto ha tenido una duración de 84 días, el coste aproximado sería de unos **3120 €**.

Análisis del sistema de información

Identificación del entorno tecnológico

El entorno tecnológico constará de un ordenador de sobremesa y máquinas virtuales.

En el lado de las tecnologías usadas, contaremos con:

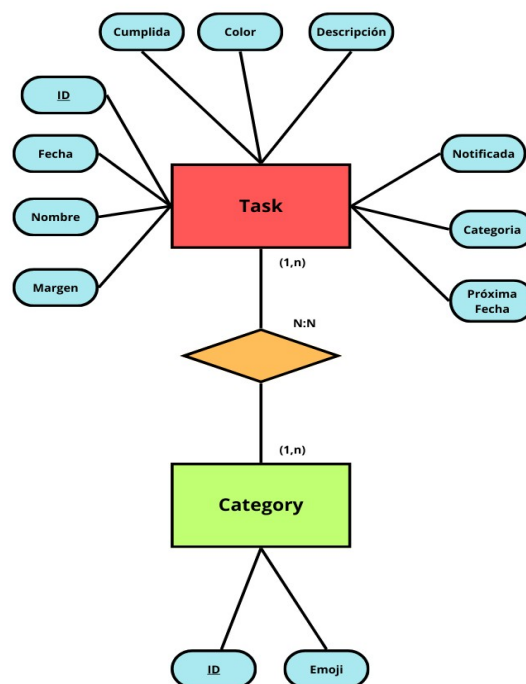
- Lenguaje de programación Kotlin → en un futuro, se espera que la aplicación tenga versión de escritorio. Kotlin tiene la capacidad de ser multiplataforma y gracias a la librería de Compose for Desktop, se podrá actualizar la aplicación para poder ser utilizada en distintos dispositivos.
Además, Kotlin, al ser el lenguaje predeterminado para Android, no para de recibir mantenimiento y actualizaciones que no paran de mejorar la sintaxis y la funcionalidad.
- Entorno de desarrollo Android Studio → es el ide por defecto para desarrollar aplicaciones nativas para Android. Nos permite lanzar máquinas virtuales de smartphones para mostrar como sería el resultado final de nuestra aplicación.
- Room → es la librería que tendremos que implementar para el uso y manejo de nuestra base de datos local. Room nos permite realizar un CRUD muy fácilmente y usa SQLite para poder interactuar con nuestra base de datos.

En resumen, Room proporciona los siguientes beneficios:

- Verificación del tiempo de compilación de las consultas en SQL
- Anotaciones de conveniencia que minimizan el código estándar repetitivo y propenso a errores
- Rutas de migración de bases de datos optimizadas

Modelo de datos

Tal como describimos en el estudio de la viabilidad, nuestro modelo de datos Entidad-Relación sería así:



La entidad Task permite tener varias categorías, y la una misma categoría puede estar en varias tareas, por lo que creamos otra tabla Task_Category al ser una relación N:N

Para hacernos una idea de la estructura, aquí tendríamos un ejemplo de varias tareas

id	Nombre	Descripción	Fecha	Margen	ProximaFecha	Color	Notificada	Cumplida	Categoría
1	Starling, cape	Fusce lacus purus, aliquet at, feugiat non, pretium quis, lectus.	117519	6098	798746	1493	0	0	3
2	Squirrel, thirteen-lined	Proin risus.	934325	9440	779948	9157	0	1	0
3	Pine siskin	Nunc rhoncus dui vel sem.	631893	8100	620056	2177	1	0	4
4	Common wolf	Vestibulum rutrum rutrum neque.	719399	3083	961265	6077	1	0	3
5	Pygmy possum	Proin leo odio, porttitor id, consequat in, consequat ut, nulla.	549360	2731	619273	1736	0	1	6
6	Antelope, roan	In hac habitasse platea dictumst.	610574	4174	978611	4655	1	2	7
7	Black-eyed bulbul	Phasellus sit amet erat.	562887	6000	241373	478	1	1	7
8	Ringtail, common	Donec vitae nisi.	488310	6173	64251	7583	0	1	1
9	Tawny frogmouth	Ut tellus.	373983	4131	102911	110	0	1	8
10	Jackal, golden	Nam dui.	357676	7536	201923	9091	0	2	4

Identificación de los Usuarios Participantes y Finales

Usuarios Finales: Al ser una aplicación a nivel individual, está enfocada a cualquier persona que quiera llevar una gestión y un seguimiento de sus tareas.

Por lo tanto, la propia persona será la administradora de la aplicación.

Identificación de Subsistemas de Análisis

Para organizar el desarrollo , se han identificado varios subsistemas clave. Estos son:

Gestión de Tareas

Este subsistema es fundamental para la funcionalidad de la aplicación, encargándose de todas las operaciones relacionadas con las tareas:

- **Creación de Tareas:** Permite a los usuarios crear nuevas tareas ingresando detalles como título, descripción, y fecha de vencimiento. El usuario deberá elegir una fecha de la tarea y un margen de tiempo; dividido en minutos, horas y días, en el que fijará la fecha límite de la tarea. Una vez haya llegado la hora de la tarea, podremos marcarla como cumplida. Una vez marcada, la tarea quedará registrada en la base de datos y podremos localizarla en el calendario de la aplicación.
- **Edición de Tareas:** Permite a los usuarios modificar los detalles de las tareas existentes.
- **Eliminación de Tareas:** Proporciona la capacidad de eliminar tareas que ya no son relevantes o necesarias. Esta función ayuda a mantener la lista de tareas organizada y actualizada.
- **Listado de Tareas:** Muestra todas las tareas creadas por el usuario, ordenadas por fecha de creación. La lista de tareas es la vista principal donde los usuarios pueden ver y gestionar sus tareas diarias.

Notificaciones

El subsistema de notificaciones se encarga de alertar a los usuarios sobre las tareas pendientes o próximas a vencerse:

- **Configuración de Notificaciones:** Configura las notificaciones para que se activen en los momentos adecuados. Cuando se cree una tarea, iniciará un cuenta atrás en segundo plano, para notificar al usuario, a la hora designada de la tarea

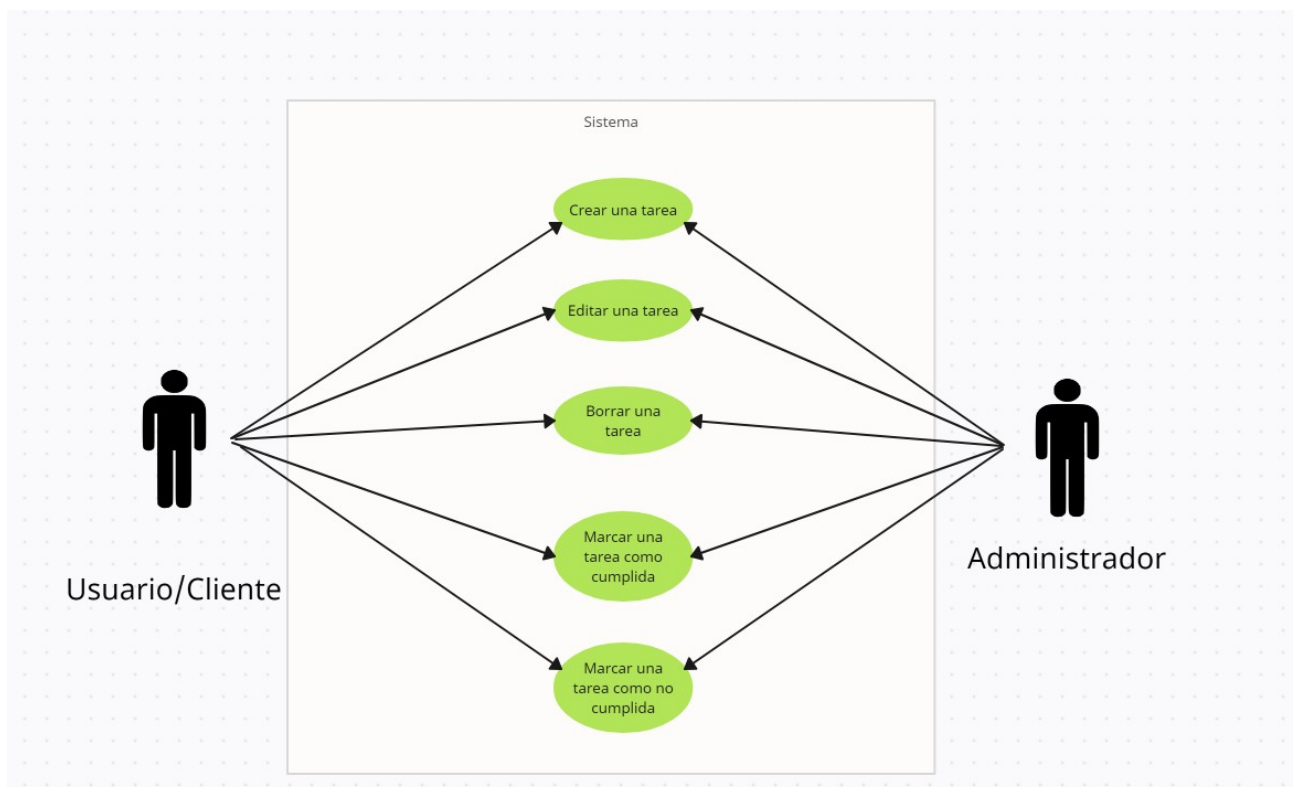
Seguimiento Temporal de Tareas

Dado que la aplicación se centra en el seguimiento de tareas a lo largo del tiempo, es importante destacar este aspecto:

Historial de Tareas: Mantiene un registro de todas las tareas completadas y eliminadas y por completar. Esto permite a los usuarios revisar sus actividades pasadas y evaluar su productividad a lo largo del tiempo.

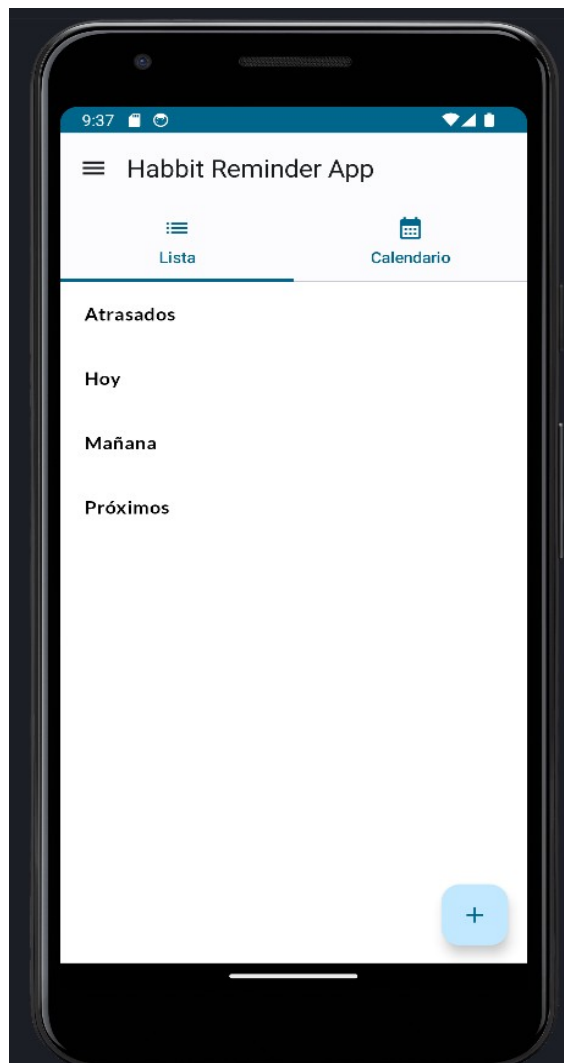
Visualización de Progreso: Muestra el progreso del usuario en la gestión de sus tareas a lo largo del tiempo. Esto se refleja en la vista de calendario dentro de la aplicación, donde nos permite ver, según diferentes tonalidades de color, la cantidad de tareas completadas de cada día.

Diagrama de casos de uso



Definición de Interfaces de Usuario y guía de uso

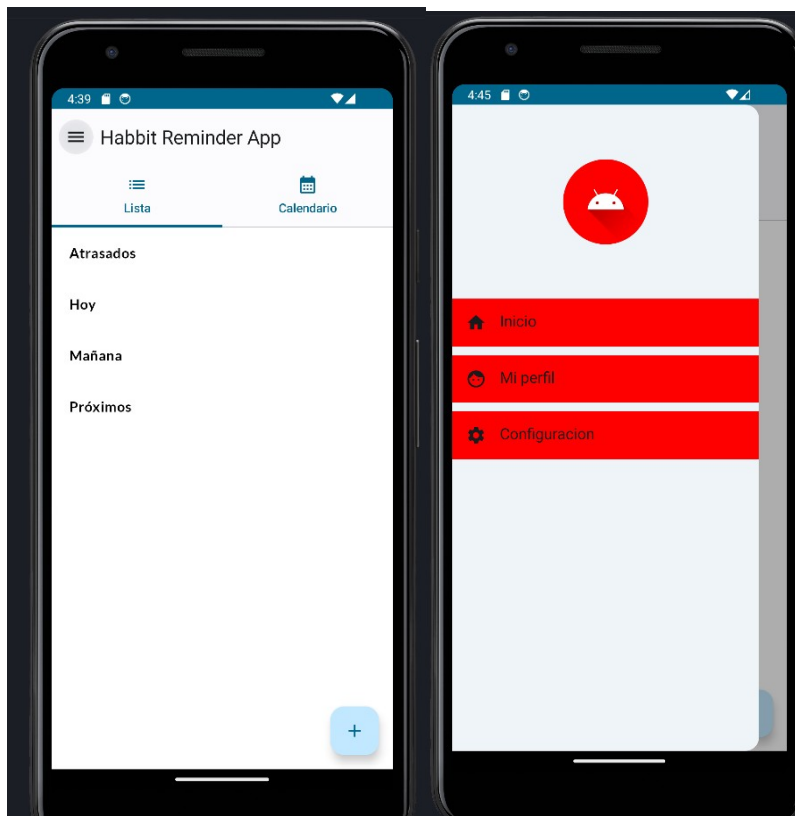
Pantalla principal



La pantalla principal se divide en 2 vistas

- Lista → Esta pantalla nos permite visualizar todas nuestras tareas disponibles, dispuestas en una columna, en la que están separadas por secciones:

- Las tareas atrasadas
- Las tareas de hoy
- Las tareas de mañana
- Las próximas tareas
- Calendario → Esta pantalla nos mostrará un calendario mensual, cuyos días estarán tonalizados de un color en concreto, en función del número de tareas marcadas como completadas.



En la parte superior izquierda de la pantalla, podremos abrir un desplegable, ofreciéndonos un menú de navegabilidad entre las distintas pantallas, entre las que se encuentran:

- Inicio → Pantalla principal
- Mi perfil → Donde veremos el numero de tareas cumplidas y el numero de tareas actuales
- Configuración → Nos permitirá entre otras opciones, cambiar el tema de la aplicación, el idioma, el tamaño de fuente, etc.

Pantalla de nueva tarea

Si desde la pantalla principal pulsamos en el botón flotante de la parte inferior derecha, nos mandará a la pantalla de creación de tarea

El nombre que tendrá la tarea y será introducido en la base de datos

Descripción opcional para cada tarea

En estos 3 campos de texto, nos permitirá elegir el rango de tiempo hasta la próxima notificación de la tarea

Nombre de la tarea

Selección de categoría

Elige color

Descripción

Fecha de inicio

La fecha seleccionada es

Siguiete fecha cada:

Minutos

Horas

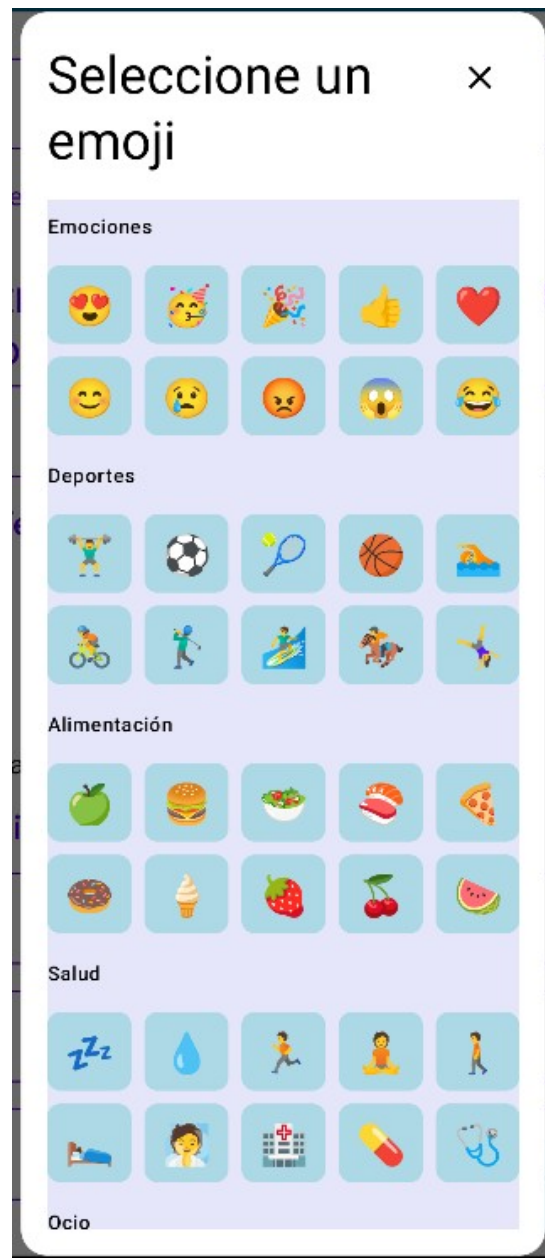
Días

La categoría se elegirá en base al emoji elegido

El color del contenedor de la tarea

Al pulsar en el calendario, se nos desplegará un calendario y un reloj para elegir la fecha y hora

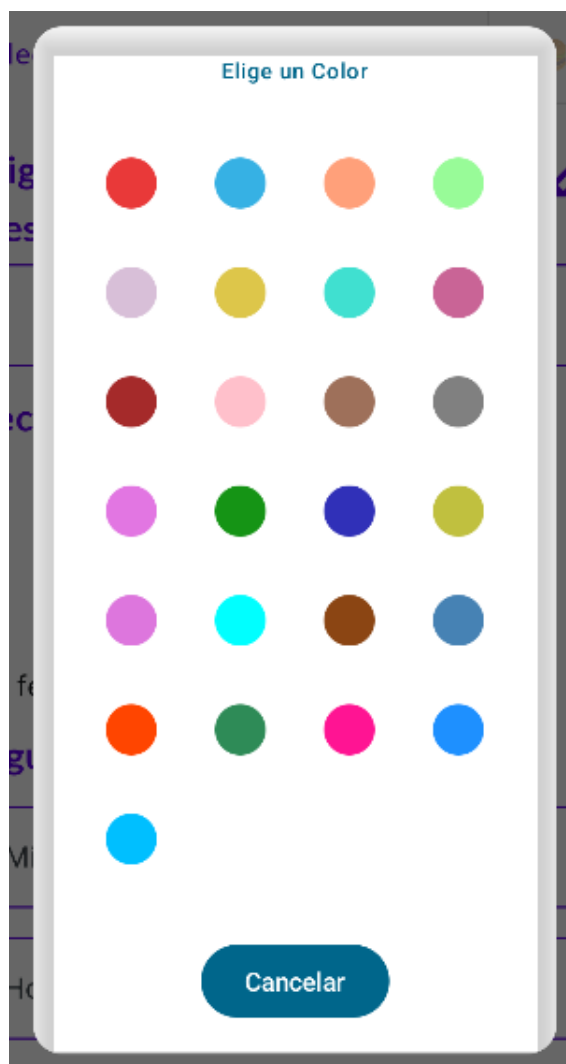
Crearemos la tarea cuando rellenemos todos los cambios

Selección de categoría

Al pulsar en el emoji de la categoría, nos permitirá elegir entre una lista de mojinás que servirá para catalogar nuestra tarea y así encontrarla más fácilmente entre nuestra colección de tareas.

Selección de color

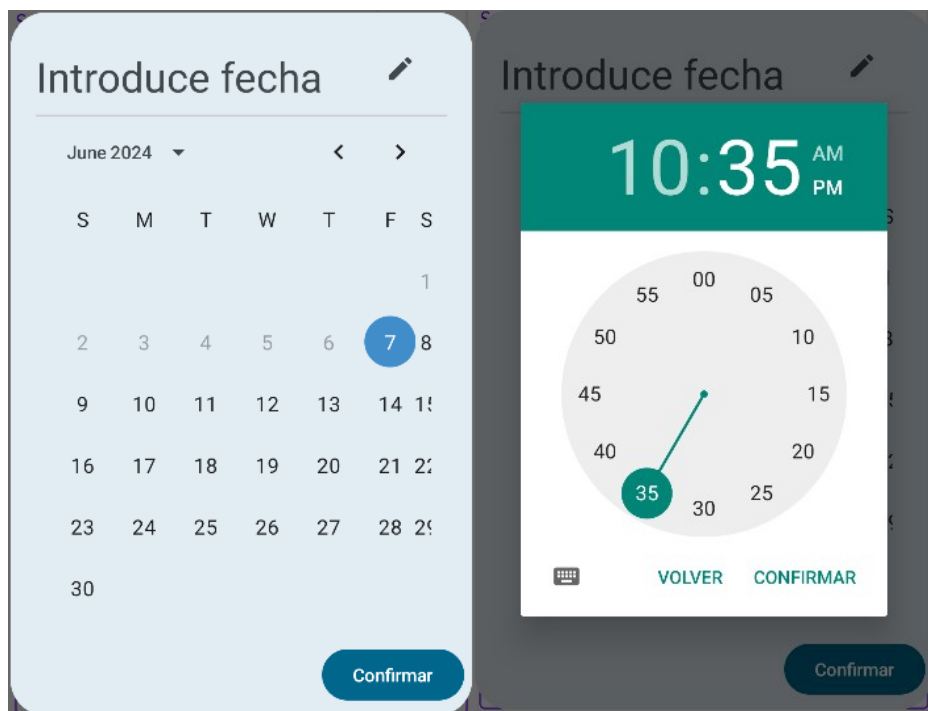
Al igual que la selección de categoría, el pulsar sobre el cuenta gotas nos abrirá una ventana donde elegir el color de nuestra tarea y poder personalizar nuestra tarjeta.



Selección de fecha

Al clicar sobre el calendario, se nos mostrará uno en formato mes, que nos permitirá elegir el día en el que comenzaremos la tarea. No se nos permitirá crear la tarea en días anteriores al nuestro.

Tras confirmar nuestra opción, se nos mostrará un reloj, en el que tendremos que escoger la hora del día.



Por último, elegiremos el rango de tiempo que queremos que nos envíen la notificación entre minutos, horas y días.

Siguiente fecha cada:

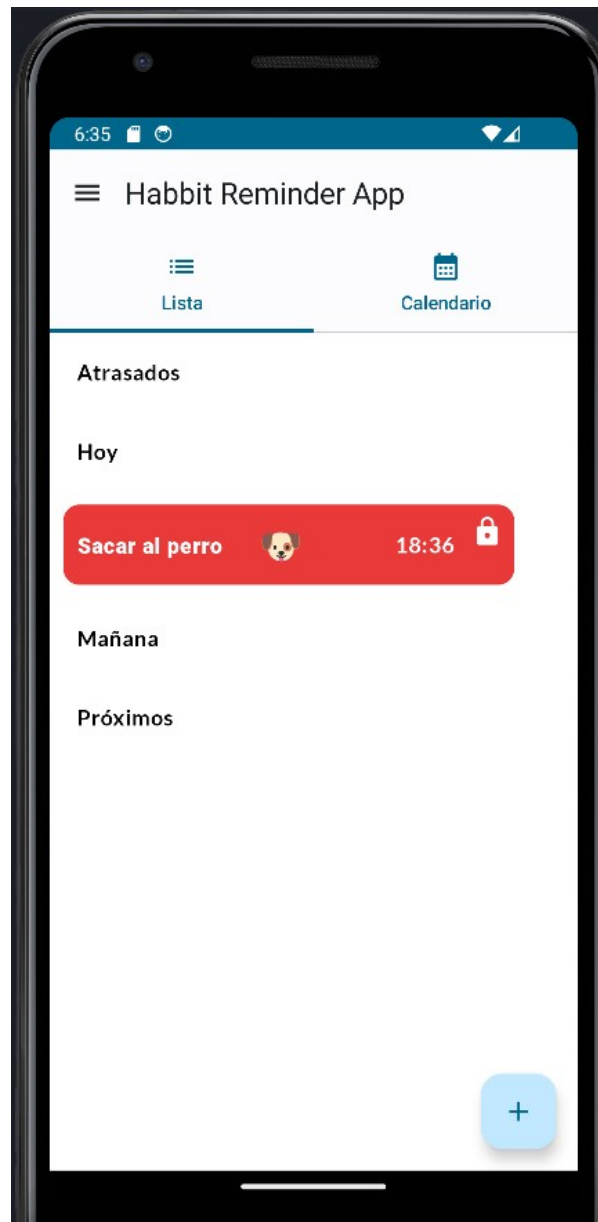
Minutos

Horas

Días

✓

Tras completar todos los pasos, al pulsar en el botón flotante, tras haber rellenado todos los campos, habremos creado nuestra tarea, y volveremos a la pantalla principal.



Ahora tendremos nuestra tarea situada, dependiendo de cuándo la hayamos creado, en una sección específica.

En la tarjeta de nuestra tarea, podemos observar el nombre de la tarea, el emoji seleccionado, la hora de la tarea, y si la tarea puede ser marcada como cumplida, como pasada, o dentro de la fecha.

Tipos de tarea

Podemos diferenciar 3 estados de las tareas:

- **En espera/bloqueada** → la tarea se encuentra en este estado, cuando la fecha actual, es menor a la de la propia tarea, podemos distinguir este estado, cuando la tarea tiene este icono en la parte derecha



- **En activo/desbloqueada** → la tarea se encuentra en este estado, cuando la fecha actual, es igual o mayor a la de la propia tarea. El icono pasaría a ser este:



- **Fuera de fecha/cancelada** → la tarea se encuentra en este estado, cuando la fecha actual es igual o mayor a la fecha más el margen de la tarea, que sería el equivalente a la siguiente fecha de la tarea. El icono pasaría a ser este:



Así tendremos una lista de tareas con diferentes estados



Interacciones con las tareas

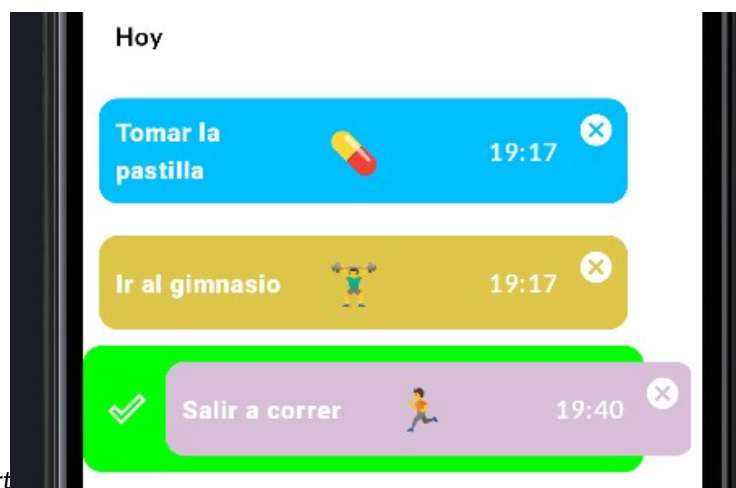
Se ha implementado swipes para manejar las tareas en las listas de la pantalla principal con diversas funcionalidades.

Dependiendo del estado de la tarea, se encuentran:

- Si la categoría se encuentra en estado bloqueada/espera, solo podremos hacer swipe a la izquierda. El swipe a la izquierda realiza la función de eliminar la tarea de la base de datos y **está disponible en cualquier estado.**



- Si la categoría se encuentra en estado “en activo” o “fuera de fecha” tendremos la posibilidad de, al igual que hacer swipe a la izquierda para eliminarla, de hacerlo a la derecha. Si está en activo, la tarea se marcará como **Cumplida** y en la base de datos, el campo cumplida será 1. En el otro caso, la tarea se marcará como **Fuera de fecha** y el campo cumplida será 2.



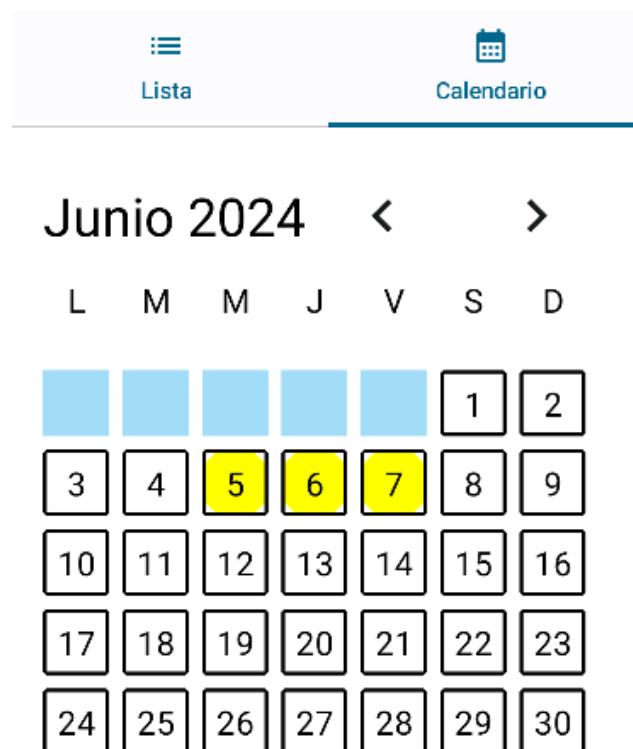
Tras marcar como cumplida o fuera de fecha, se creará una nueva tarea, con los mismos datos, exceptuando la fecha, cuya tal será la fecha previa, más el margen de tiempo.

Por ejemplo, tenemos una tarea a las 21:30 con un margen de 30 minutos.

Si marcamos como cumplida la tarea, una vez que la hora sea mayor que las 21:30, se creará otra tarea a las 22:00.

Ventana de calendario

Si deslizamos en la pantalla principal hacia la derecha, nos encontraremos con la ventana del calendario mensual.



Esta ventana nos permite llevar un seguimiento de nuestras tareas a lo largo del tiempo. Según el color del día, podemos distinguirlos:

- Blancos → No hay tareas para ese día
- Amarillos → Se ha cumplido alguna tarea, pero no todas
- Rojos → No se ha cumplido ninguna tarea programada ese día
- Verde → Se han cumplido todas las tareas ese día

Podremos navegar entre los meses pulsando en cada flecha.

Si pulsamos en uno de los días donde hayan tareas, se nos abrirá una lista, compuesta por todas las tareas de ese día



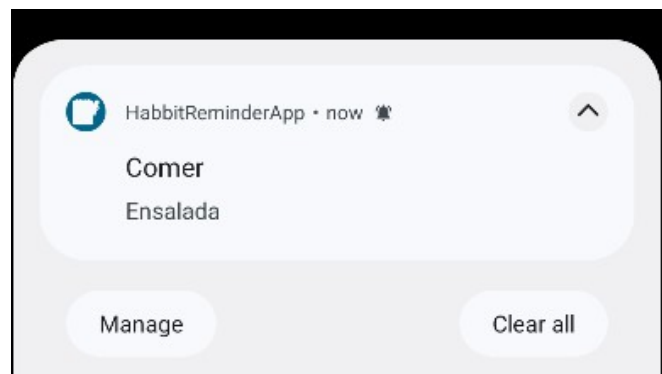
A la derecha de cada tarea, nos encontraremos con tres tipos de icono, según si la tarea ha sido marcada como **Fuera de fecha**, **Cumplida** o **En activo**.

Notificaciones

La aplicación mandará una notificación a nuestro dispositivo móvil cuando llegue la hora de una de las tareas definidas, con el nombre de nuestra tarea, y la descripción de la misma. Para ello, previamente deberemos haber dado nuestro permiso a recibir notificaciones.

La notificación se cancelará si eliminamos nuestra tarea.

Al pulsar sobre esta, abriremos la aplicación.



Manual de instalación

Para poder descargarnos el proyecto, hemos creado un repositorio en github, donde podemos recoger la APK de la aplicación.

El enlace al repositorio de github es el siguiente:

<https://github.com/Driusito/HabbitReminderApp>

Tan solo debemos irnos a las releases del repositorio, para tener acceso a la apk del proyecto.

Conclusiones

Con este proyecto, he realizado un trabajo de investigación bastante extenso, al trabajar con múltiples tecnologías nuevas, tales como:

- Jetpack Compose → que cambia drásticamente la manera de programar, ya que, previamente, utilizabamos XML para desarrollar nuestras aplicaciones. Compose hace que el desarrollo sea mucho más rápido y también mejora la calidad y legibilidad del código.
- Dagger Hilt → librería que nos simplifica la inyección de dependencias. No es imprescindible para este proyecto en su primera version, pero al aumentar las especificaciones y funcionalidades del proyecto, simplificará mucho el código.
- WorkManager → una librería que se encarga de manejar los procesos en segundo plano. Esta ha sido imprescindible para la gestión de las notificaciones
- Swipe → al querer darle un diseño más amigable, se ha optado por la interacción mediante deslizamientos en las tareas, por lo que se ha requerido de una librería encargada de hacer swipes.
- Room → es la biblioteca encargada de la persistencia de datos de forma local. Utiliza SQLite para manejar la base de datos.

Bibliografía

WorkManager

<https://developer.android.com/codelabs/android-workmanager?hl=es-419#0>

Room

<https://developer.android.com/jetpack/androidx/releases/room?hl=es-419>

Jetpack Compose

<https://developer.android.com/develop/ui/compose/tutorial?hl=es-419>

Inyección de dependencias con Dagger Hilt

<https://developer.android.com/training/dependency-injection/hilt-jetpack?hl=es-419>